AI Copilot Usage Log (Curated)

Project: OzZoo GUI | Session window: 2026-04-06 to 2026-04-10

Generated: 2026-04-10 21:13:14

This log is curated for major design, generation, debugging, and optimisation steps; it is not a full transcript.

1. Debugging feed_animal object-mapping mismatch

Key Prompt: I cant seem to feed the animals in the simulation game ... TypeError: Food() argument after ** must be a mapping, not Food

AI Output Snippet:

```
# zoo.py
food = FOOD_TYPES.get(food_type)
if not food:
    return False, f"Unknown food type: {food_type}"
if animal.feed(food):
    self._food_stock[food_type] -= 1
```

Your Analysis: This prompt was used to diagnose a traceback-first bug. The fix replaced dictionary unpacking with direct object usage because FOOD_TYPES stores Food instances, not dictionaries.

2. Fixing shop tab crashes for Food and Medicine objects

Key Prompt: shop is empty (not displaying) ... TypeError: 'Food' object is not subscriptable

AI Output Snippet:

```
# shop.py
for food_type, food in FOOD_TYPES.items():
    self._create_purchase_row(..., food.name, food.cost, "food")

for med_type, medicine in MEDICINE_TYPES.items():
    treats = ", ".join(medicine.treats)
    self._create_purchase_row(..., f"{medicine.name} (treats: {treats})",
                              medicine.cost, "medicine")
```

Your Analysis: I used this prompt to unify data access patterns in the shop UI. Both tabs now read resource objects through attributes, which removed subscriptability errors and restored rendering.

3. Gameplay rule: full animals refuse food

Key Prompt: can we make the animals refuse to eat when they have 0% hunger

AI Output Snippet:

```
# animal.py
if not self._is_alive:
    return False
if self._hunger == 0:
    return False
if food.food_type != self.get_diet():
    return False
```

Your Analysis: This prompt introduced a clear game rule and prevented unrealistic overfeeding. The guard clause was added before diet checks so refusal happens immediately when hunger is already zero.

4. Behaviour tuning: escape chance scales with unhappiness

Key Prompt: make it a higher probability the unhappier the animal becomes when lower than 30%

AI Output Snippet:

```
# animal.py
if self.is_stressed:
    escape_probability = (30 - self._happiness) / 100
    if random.random() < escape_probability:
        self._is_alive = False
        self._notify_observers("escape")
```

Your Analysis: I used this prompt to implement a proportional risk mechanic. The formula increases escape likelihood as happiness approaches zero, which creates a meaningful welfare-management consequence.

5. Adding in-game reset path from Settings

Key Prompt: now I want to make an option to start a fresh game from scratch

AI Output Snippet:

```
# settings.py
def _on_new_game(self) -> None:
    if self._app.ask_yes_no("New Game", "... progress will be lost ..."):
        zoo = create_default_zoo()
        self._app.zoo = zoo
        self._app.set_status("New game started! Welcome to OzZoo!")
        self._app.show_frame("zoo_status")
```

Your Analysis: This prompt targeted UX flow. The solution gave players a one-click reset without restarting the app, while still protecting them with a confirmation dialog.

6. Designing and implementing a map-based builder

Key Prompt: add a new frame with a mini map, where enclosures can be bought and placed with the scenery and shops

AI Output Snippet:

```
# map_tile.py + zoo.py + map_view.py
class TileType(Enum):
    EMPTY = "empty"
    ENCLOSURE = "enclosure"
    SCENERY = "scenery"
    SHOP = "shop"

def place_item(self, x, y, item_id, tile_type, enclosure_ref=None):
    cost = PLACEMENT_COSTS.get(item_id, 0)
    if cost > self._budget:
        return False, f"Insufficient funds! Need ${cost:,.2f}"
```

Your Analysis: I used this prompt for feature-level implementation: model, UI frame, and budget-integrated placement logic. The work established a persistent 10x10 grid and enabled tile-based world building.

7. Pre-generating starter map layout for new games

Key Prompt: cool, pre-generate the map with the starting enclosures, trees, and shop

AI Output Snippet:

```
# zoo.py
def create_default_zoo() -> Zoo:
    ...
    _populate_mission_starter_map(zoo, required_habitat, enc_id)

def _populate_mission_starter_map(zoo, habitat_type, enclosure_id):
    zoo.place_item(4, 8, "ticket-booth", TileType.SHOP)
    zoo.place_item(3, 8, "food-stall", TileType.SHOP)
    zoo.place_item(5, 8, "bench", TileType.SHOP)
```

Your Analysis: This prompt improved onboarding by giving a visible starter layout immediately. It reduced blank-map friction and aligned map state with initial gameplay context.

8. Batch bug-fix pass from traceback bundle

Key Prompt: the map frame budget isnt updated dynamically ... cannot purchase food or medicine ... cannot confirm purchase of an animal

AI Output Snippet:

```
# zoo.py
food = FOOD_TYPES[food_type]
total_cost = food.cost * quantity
medicine = MEDICINE_TYPES[medicine_type]
total_cost = medicine.cost * quantity

# map_view.py
def _update_budget_display(self) -> None:
    self._budget_label.config(text=f"Budget: ${self._zoo.budget:,.2f}")
```

Your Analysis: I used this as a traceback-driven stabilisation prompt. Fixes focused on object attribute access, budget UI refresh wiring, and purchase flow reliability.